

Utilities & Legacy Commands Reference (Midas_V2 v0.3.21)

****Last updated:**** 2025-09-20 15:28 (America/Chicago)

Your quick reference for the helper scripts we've used or mentioned. Use these as needed—no workflow change implied.

1) Clear outputs safely ****Purpose:**** Wipe generated outputs in `out\` when you want to re-run clean, without touching `data\` or `config\`.

````powershell python scripts/clear_out.py ```` - Typical behavior: removes `out\\*` folders/files, preserves data and configs. - If you want to keep certain subfolders, run a targeted clear (example pattern): ````powershell python -c "import shutil, pathlib; p=pathlib.Path('out/20250805'); shutil.rmtree(p, ignore_errors=True)" ````

---

**## 2) Rebuild or repair summaries** If a summary is missing or stale:

**\*\*Range-level rebuild (safe):\*\*** ````powershell python scripts/run_range_safe.py --start 2025-08-05 --end 2025-08-31 --scenario B ```` - Runs missing days and **\*\*rebuilds summaries\*\*** without clobbering valid results.

**\*\*Manual summary rebuild (if you have a dedicated script):\*\*** ````powershell python scripts/rebuild_range_summary.py --start 2025-08-05 --end 2025-08-31 --scenario B ````

---

**## 3) Day runners** **\*\*Normal single day:\*\*** ````powershell python scripts/run_day_simple.py --date 2025-08-06 --scenario B ````

**\*\*Catalyst-focused day (uses catalyst-scored universe):\*\*** ````powershell python scripts/run_day_catalyst.py --date 2025-08-06 --scenario B --universe data\universe_active.txt ````

---

**## 4) Universe & data helpers** **\*\*Top gappers / daily universe build:\*\*** ````powershell python scripts/topgappers.py --date 2025-08-06 ```` - Writes a **\*\*per-day universe file\*\*** (e.g., `data\universe\_topgappers\_2025-08-06.txt`) and updates the active path your runners read.

**\*\*Fetch minute bars (Polygon), by session:\*\*** ````powershell python scripts/fetch_minutes_polygon.py --date 2025-08-06 --session rth ````

**\*\*Trim current universe to Top N (Python-only):\*\*** ````powershell python -c "import io; p='data\universe_active.txt'; s=io.open(p,'r',encoding='utf-8').read().splitlines(); io.open(p,'w',encoding='utf-8').write('\n'.join(s[:10])+'\n')" ````

---

**## 5) Inspect & analyze helpers** **\*\*Show the latest range file paths quickly:\*\*** ````powershell`

```
python scripts/show_latest_range.py --scenario B ```
```

```
Explain analysis for a given range (verbose, step-by-step): ```powershell python
scripts/analyze_range_explained.py --start 2025-08-05 --end 2025-08-31 --scenario B ```
```

```
Analyze base only (no experimental extras): ```powershell python
scripts/analyze_base_only.py --start 2025-08-05 --end 2025-08-31 --scenario B ```
```

---

```
6) Catalyst enrichment (news scoring) **Score and tag the universe by catalyst strength:**
```powershell python scripts/enrich_universe_catalyst.py --date 2025-08-06 ``` - Start by  
whitelisting **A-grade** tickers (highest score). - Allow **B-grade** only to fill a modest  
Top■N on slow days.
```

```
## 7) Quick sanity checks - Confirm a specific day's results exist: ```powershell python -c  
"import pathlib; p=pathlib.Path('out/20250806/D/results_2025-08-06.csv'); print(p.exists())"  
``` - Peek first lines: ```powershell python -c "import pathlib;  
p=pathlib.Path('out/20250806/D/results_2025-08-06.csv');
print('\n'.join(p.read_text(encoding='utf-8').splitlines()[:10]) if p.exists() else 'no
results')" ```
```

---

```
8) Undo & restore (Git) - Revert a universe file change (if tracked by Git): ```powershell
git checkout -- data\universe_active.txt ``` - Tag after a clean run: ```powershell git tag -a
v0.3.22-aug-baseline -m "August baseline completed (B,D,E)"; git push; git push --tags ```
```

**Reminder:** Keep **one change at a time** (Top■N OR RVOL OR catalysts) so your comparisons are trustworthy.